# FeathersJS MCP Server

## MVP Presentation

CPSC 319 · Team B · March 2026

Client: FeathersJS (David Luecke)


| | |
|---|---|
| **Nariman** | Client Liaison |
| **Nam** | Technical Writer |
| **Maharaj** | DevOps |
| **Nazif** | Project Manager |
| **Maggie** | UX |


Course Instructor: Parsa Rajabi · TA: Stash Currie

# Today's Agenda

1. Problem & MVP Scope
2. Requirements & Architecture
3. Live Demo
4. Reflection & Next Steps
5. Summary & Q&A

# Past Vs Future

Terminal

```
> Do you want to use JavaScript or TypeScript? TypeScript
> Write a short description A Feathers chat application
> Which HTTP framework do you want to use? KoaJS (recommended)
> What APIs do you want to offer? HTTP (REST), Real-time
> Which package manager are you using? npm
> Generate end-to-end typed client? Yes
> What is your preferred schema (model) definition format? TypeBox (recommended)
> Which database are you connecting to? SQLite
> Enter your database connection string feathers-chat.sqlite
```

Can u help me setup the feathers chat app using mcp

feathersMCP - resolve-library-id ✓ Success Re-run
How to setup a chat app with FeathersJS, FeathersMCP

feathersMCP - query-docs ✓ Success Re-run
/@feathers/feathers, How to create a chat application step by step using the Feathers CLI and generator

feathersMCP - query-docs ✓ Success Re-run
/@feathers/feathers, FeathersJS chat tutorial messages service schema and hooks

Setting up a Feathers chat application is a great way to learn the framework's real-time capabilities. Based on the latest Feathers (Dove) documentation, here is the step-by-step guide to building the backend.

**1. Initialize the Project**

First, create a new Feathers application using the CLI. This will prompt you for configuration details (Language: TypeScript/JavaScript, Database: SQLite/PostgreSQL/etc.).

bash

```bash
npm create feathers@latest feathers-chat
cd feathers-chat
```

**2. Generate Authentication**

Feathers makes authentication easy. Run the following command to set up the user service and authentication strategy (usually JWT and Local).

Enter a message here, press ⏎ to send

0.6%

**OLD WAY ❌**

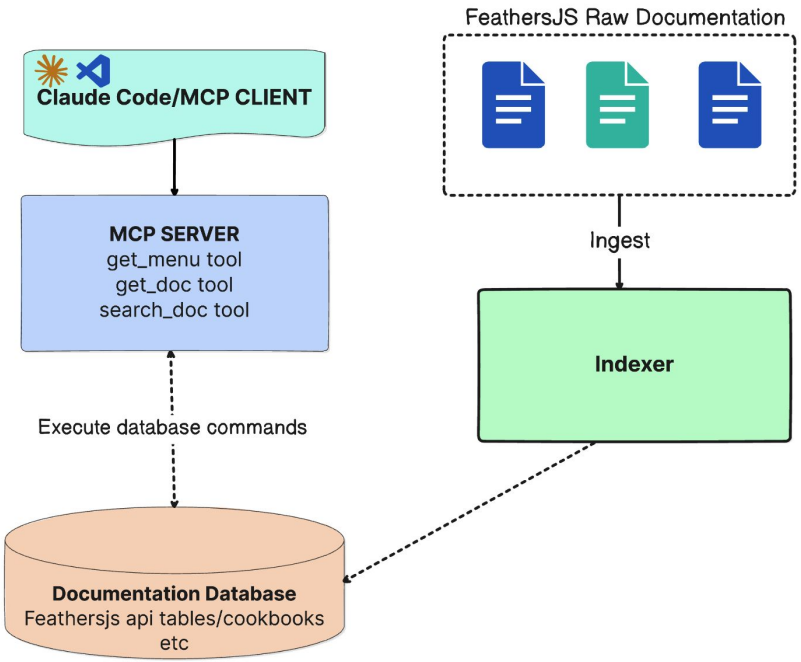
**NEW WAY ✅**

# MVP Scope

| Implemented (Must-Have) | Deferred (Out of Scope) | Changed Since Design |
|---|---|---|
| MCP documentation server | Community knowledge base for contributed posts | Removed service scaffolding tool |
| Full-text search across 47 FeathersJS v6 docs | Community content submission via GitHub PR | LLM-written SQL → keyword-based search (2 tools) |
| 4 documentation query tools | Automated review-to-publish pipeline | Removed Backend API layer |
| Pre-built documentation ingestion pipeline | Expanded documentation coverage | |
| Bundled pre-built database | Automated documentation updates | |
| Works with VS Code, Claude Desktop, Cursor | | |
| Clean, noise-free documentation responses | | |

*All changes driven by: zero operational cost · bundled database · deterministic responses*

# High-Level Design




## Request Lifecycle

1. Developer asks a question
2. Agent communicates with our MCP
3. MCP interacts with the database
4. Results returned to Agent for reasoning

# UX with Gemini CLI

```
setings.json                                JSON

  "mcpServers": {
    "feathermcp": {
      "command": "node",
      "args":
      ["FeatherMCP/build/index.js"]
    }
```

**Add this configuration to agent settings**

**Ask a question about feathersjs, and the agent discovers our MCP**

```
> gemini

GEMINI

Logged in with Google: nzfishzak60@gmail.com /auth
Plan: Gemini Code Assist in Google One AI Pro

 > explain me how feathersjs hooks work

✦ I will start by exploring the FeathersJS documentation to provide a comprehensive explanation of how hooks work.

 Action Required

 ?  get-menu (feathermcp MCP Server) {}

 MCP Server: feathermcp
 Tool: get-menu

 MCP Tool Details:
 (press Ctrl+O to expand MCP tool details)
 Allow execution of MCP tool "get-menu" from server "feathermcp"?

   1. Allow once
 ● 2. Allow tool for this session
   3. Allow all server tools for this session
   4. No, suggest changes (esc)
```

**Get context enriched response from your agent**

```
✦ FeathersJS hooks are pluggable middleware functions that you can register around, before, after, or on errors of a service method. They are a core
  feature of Feathers that allow you to separate your business logic from your data storage logic.


  1. How They Work
  Hooks are transport independent, meaning they run the same way whether a request comes from a REST API, WebSockets, or an internal server call. They
  are also service agnostic, allowing the same hook to be reused across different services (e.g., a "logging" hook used by both users and messages).

  2. Types of Hooks
  There are four types of hooks, executed in a specific order:


   1. Around Hooks (around): Introduced in Feathers v5 (Dove), these wrap around the entire service call. They use an await next() call to control when
      the service method and other hooks execute.


   1     async (context, next) => {
   2       console.log('Before the service call');
   3       await next();
   4       console.log('After the service call');
   5     }
   2. Before Hooks (before): Run before the service method. Commonly used for validation, permission checks, or modifying incoming data.
   3. After Hooks (after): Run after the service method successfully completes. Used for logging, sending emails, or filtering sensitive data from the
      result.
   4. Error Hooks (error): Run only if an error is thrown during the process. Used for centralized error logging or transforming error messages.


  3. Execution Flow
  When a service method (like find or create) is called, hooks execute in this order:
    1. around (code before await next())
    2. before
    3. The service method (e.g., the database query)
    4. after
    5. around (code after await next())

  If an error occurs at any point, the flow jumps straight to the error hooks.


  4. The Hook Context
  Every hook receives a context object containing information about the current call:
    * context.app: The Feathers application object.
    * context.service: The service instance.
    * context.method: The method name (find, get, create, etc.).
```

# Requirements → Features → Demo

| Requirement | → | Feature Built | → | Demo Evidence |
|---|---|---|---|---|
| US-001<br>**Retrieve up-to-date FeathersJS documentation**<br>*User story: As a new FeathersJS user, I want to learn FeathersJS concepts* | → | Documentation search and retrieval across 47 v6 pages | → | **Live demo — Copilot answers a FeathersJS concept question** |
| US-002<br>A user wants to create a full FeathersJS application | → | Application setup and service pattern documentation | → | **Pre-demo evidence: we used our FeathersJS MCP to scaffold a full FeathersJS application end-to-end** |
| FR-005-6<br>Adding new features to our FeathersJS application | → | Hooks, authentication, and extension documentation | → | **Live demo — ask Copilot how to add authentication or a new hook** |

# Requirements Coverage

| ID | Requirement | Status | Feature Built |
|---|---|---|---|
| US-001 | Users can get answers about FeathersJS documentation | Done | search-doc + get-doc |
| FR-002 | Documentation retrieval based on natural language | Done | MCP + search-doc |
| NFR-001 | Response < 2 seconds | Done | ~50ms SQLite in-process |
| NFR-006 | Setup time < 15 minutes to full app | Done | Bundled DB + no external services |
| NFR-012 | Zero operational cost / within free-tier limits | Done | Bundled 4.6 MB SQLite |
| FR-003 | Developer can scaffold a service via tools | Done | Retrieval-only MVP, LLM writes code |
| US-010 | Community can submits content / PR to knowledge base | Partial | Requires Supabase + GitHub PR |
| FR-009 | Submit Q&A entry to knowledge base | Partial | Depends on blog DB |

# Live Demo — GitHub Copilot + MCP

Copilot is connected to our MCP server — watch it pull live FeathersJS v6 docs in real time

## GitHub Copilot

**Asks questions in natural language**
No FeathersJS knowledge required

**Calls MCP tools automatically**
search-doc, get-doc, get-schema, get-menu

**Answers grounded in real docs**
No hallucinated or outdated v5 responses

↔

## FeathersJS MCP Server

**47 FeathersJS v6 docs indexed**
SQLite + FTS5 full-text search (BM25)

**Runs locally, zero cost**
Offline, deterministic, no API key

**Pre-built 4.6 MB bundled database**
Ships with the npm package, ready to use

## ★ FEATURED DEMO QUESTION

*"What are around hooks in FeathersJS?"*

### Without MCP

what are around hooks in featherjs. do not use mcp. let me know if it is not a thing

✓ Considered "around hooks" in FeatherJS and their implications

"Around hooks" are not a standard feature in FeatherJS.

FeatherJS has three main types of hooks:

1. **before hooks** - Execute before a service method
2. **after hooks** - Execute after a service method
3. **error hooks** - Execute when an error occurs

You might be thinking of "around" hooks from other frameworks (like aspect-oriented programming), but FeatherJS doesn't have this hook type. However, you

### With MCP

what are around hooks in featherjs. Let me know if it does not exist

✓ Searched for information about around hooks in FeatherJS

I'll search for information about "around hooks" in FeatherJS documentation.

✓ Searched for "around hooks" in documentation

✓ Ran `search-doc` – featherjs-docs (MCP Server)

✓ Analyzed around hooks feature in FeatherJS documentation

Around hooks **do exist** in FeatherJS. They were introduced in Feathers v5 (Dove).

Around hooks are middleware functions that wrap around other hooks and the service method. They use a `next()` function pattern:

- Code **before** `await next()` runs before other hooks and the service method
- Code **after** `await next()` runs after everything completes
- They can handle errors with try/catch blocks

**Example:**

```
app.service('messages').hooks({
  around: {
    all: [
      async (context: HookContext, next: Nextf
        const startTime = Date.now()

        await next() // Execute other hooks ar
```

# What We Learned

01

## CSS Noise Was Invisible Until Testing

- 39 of 47 docs had CSS artifacts
- Caught in testing, not dev

02

## The HTTP Layer Added No Value

- Local process needs no API
- Cut entirely, simpler + free

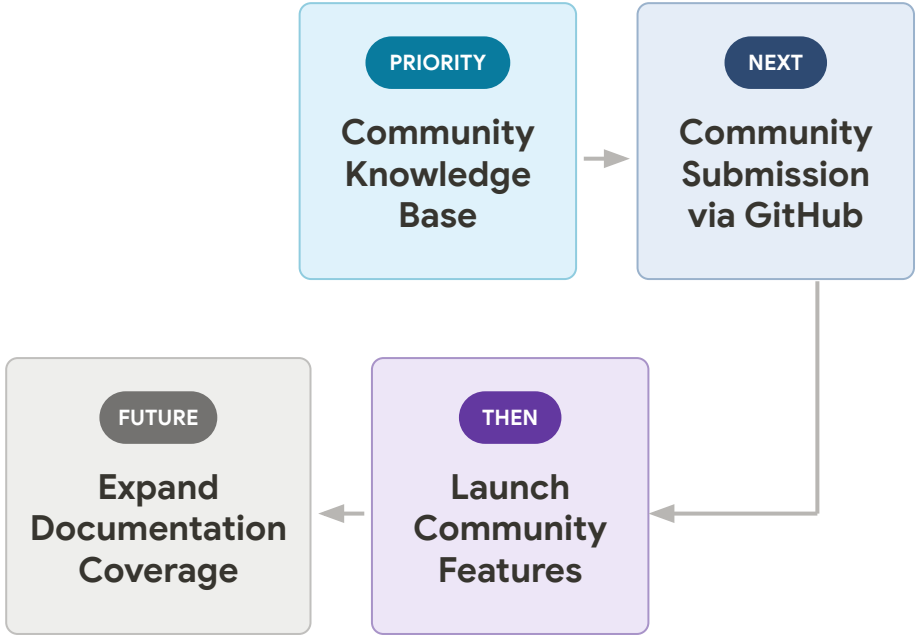
03

## Truncation Was Critical for Quality

- 15K chars × 5 overflowed context
- Led to two-tier retrieval

# What’s Next

ROADMAP




KEY RISKS

## Hosting Cost

- Community features run on Supabase free tier

## Content Quality

- Submissions need moderation

## Context Window

- More docs = longer responses; managed with content limits

# Summary & Questions

WHAT WE DEMONSTRATED

01 Working MCP server — 4 tools, 47 real docs

02 Live feature implementation in demo

03 Edge cases and context limits handled

04 Zero cost · IDE-ready

OUR TEAM

## FeatherJS B

Nariman

Nam

Maharaj

Nazif

Maggie